Name - Chaitanya Kelkar

Roll no. - 53

Experiment 10

Aim - Explore and implement Autoencoders for image denoising

2. Dataset Source & Description

- **Source:** MNIST Dataset (Accessed natively via TensorFlow/Keras).
- **Description:** A standard benchmark database of grayscale handwritten digits (0–9). For this self-supervised learning experiment, the original labels are discarded. The clean images act as the ground-truth targets, while artificially corrupted versions of these images act as the inputs.
- **Size:** 60,000 training images and 10,000 testing images.
- **Features:**28x28 pixel grayscale images, flattened or reshaped into a 3D tensor of (28, 28, 1).

3. Mathematical Formulation of the Algorithm

- **A. Autoencoder Architecture:** The network consists of an Encoder f(x) and a Decoder g(h). The encoder maps the input to a hidden, compressed representation $h = f(x)$. The decoder reconstructs the input from this hidden state to produce the output $\hat{x} = g(h)$.
- **B. Denoising Objective:** Instead of clean data, the input is a corrupted version of the original data: $\tilde{x} = x + \text{noise}$. The network is trained to minimize the reconstruction error between the output $\hat{x}$ and the *original, uncorrupted* input x.
- **C. Loss Function:** Because the pixel values are normalized between 0 and 1, we use Binary Cross-Entropy (BCE) to calculate the loss for each pixel:
  $L(x, \hat{x}) = -1/N \Sigma [x_i \log(\hat{x}_i) + (1 - x_i) \log(1 - \hat{x}_i)]$

4. Algorithm Limitations

- **Data Specificity:** Autoencoders learn a highly compressed representation of their specific training data. An autoencoder trained to denoise MNIST digits will completely fail if asked to denoise images of faces, animals, or natural landscapes.
- **Blurriness:** Standard autoencoders (optimizing for BCE or Mean Squared Error) often produce slightly blurry reconstructed images because the network learns to "average out" pixel uncertainties rather than generating sharp edges.
- **Computational Cost:** Deep Convolutional Autoencoders require significant computational power and specialized hardware (GPUs) to train effectively, whereas traditional mathematical noise filters (like a Gaussian or Median filter) can run instantly on a CPU.

5. Methodology / Workflow

The experiment followed a self-supervised Deep Learning pipeline:

- **Data Preprocessing:** Loaded the MNIST dataset and entirely dropped the label arrays. Reshaped the arrays to include a channel dimension and normalized all pixel values from **0–255** to a range of **0.0–1.0**.
- **Noise Injection:** Artificially corrupted both the train and test sets by adding random values drawn from a Gaussian normal distribution. Clipped the resulting array values to ensure they didn't exceed the valid image bounds of [0.0, 1.0].
- **Model Architecture:** * *Encoder:* Used Conv2D layers to extract spatial features and MaxPooling2D layers to downsample the image dimensions, creating a tight informational bottleneck.
    - *Decoder:* Used Conv2D and UpSampling2D layers to rebuild the spatial resolution back to 28×28.
- **Model Training:** Fit the model using the noisy data as the input and the clean data as the target, forcing the network to learn how to subtract the noise.

6. Performance Analysis

- **Loss Monitoring:** The binary_crossentropy loss on both the training and validation sets was monitored. A steady decrease in loss over epochs indicated that the model was successfully learning the latent structure of the numbers.
- **Visual Validation:** Unlike classification models where accuracy is a rigid percentage, generative models are often evaluated visually. Plotting the original noisy input images directly above the model's predictions proved that the autoencoder successfully separated random static from structured digit strokes.

7. Hyperparameter Tuning

- **Noise Factor:** The multiplier (e.g., **0.5**) applied to the Gaussian noise was carefully chosen. If the noise is too low, the model acts as a trivial identity mapping. If it is too high, the underlying digit structure is mathematically destroyed and cannot be recovered.
- **Bottleneck Dimensions:** Tuned the depth of the max-pooling layers. A bottleneck that is too wide allows noise to pass through to the decoder, while one that is too narrow forces the model to lose vital structural features of the digits.
- **Optimizer & Epochs:** Utilized the adam optimizer for adaptive learning rate adjustments, and tuned the batch_size (**128**) and epochs (**10**) to find the sweet spot between training speed and maximum loss convergence.

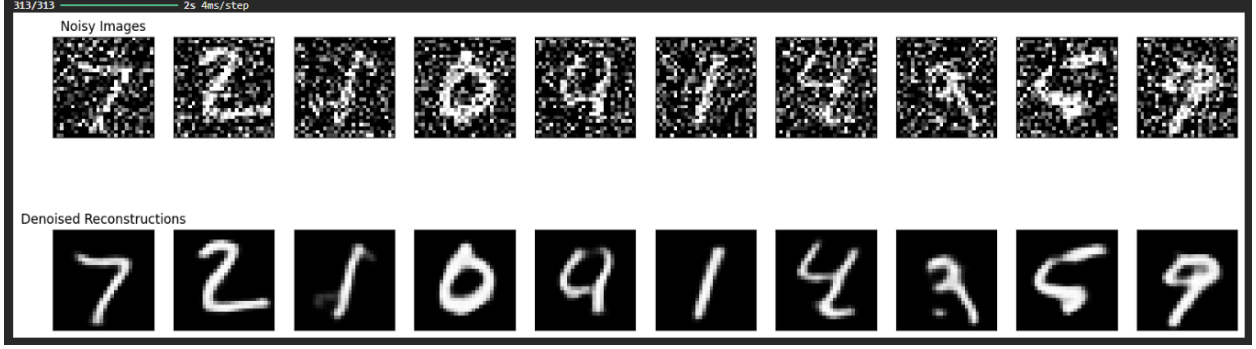
```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, UpSampling2D
from tensorflow.keras.models import Model
from tensorflow.keras.datasets import mnist

# ----------------------------------------
# Step 1: Load and Preprocess the Dataset
# ----------------------------------------
# Load MNIST (we don't need the labels for autoencoders)
```

```
(x_train, _), (x_test, _) = mnist.load_data()

# Normalize pixel values to be between 0 and 1
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.

# Reshape images to (28, 28, 1) to match CNN input shape
x_train = np.reshape(x_train, (len(x_train), 28, 28, 1))
x_test = np.reshape(x_test, (len(x_test), 28, 28, 1))

# ----------------------------------------
# Step 2: Introduce Artificial Noise
# ----------------------------------------
noise_factor = 0.5
x_train_noisy = x_train + noise_factor * np.random.normal(loc=0.0, scale=1.0,
size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(loc=0.0, scale=1.0,
size=x_test.shape)

# Clip the values so they stay in the valid [0, 1] range for images
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)

# ----------------------------------------
# Step 3: Build the Autoencoder Model
# ----------------------------------------
input_img = Input(shape=(28, 28, 1))

# ENCODER: Compress the image
x = Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
x = MaxPooling2D((2, 2), padding='same')(x) # Shrinks to 14x14
x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
encoded = MaxPooling2D((2, 2), padding='same')(x) # Shrinks to 7x7

# DECODER: Reconstruct the image
x = Conv2D(32, (3, 3), activation='relu', padding='same')(encoded)
x = UpSampling2D((2, 2))(x) # Expands to 14x14
x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
x = UpSampling2D((2, 2))(x) # Expands to 28x28
decoded = Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x) # 1 filter
for grayscale
```

```python
# Compile the model
autoencoder = Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

# ----------------------------------------
# Step 4: Train the Denoising Autoencoder
# ----------------------------------------
# CRITICAL: Input is noisy data, Target is clean data
autoencoder.fit(x_train_noisy, x_train,
                epochs=10, # Increase epochs for better results
                batch_size=128,
                shuffle=True,
                validation_data=(x_test_noisy, x_test))

# ----------------------------------------
# Step 5: Predict and Visualize Results
# ----------------------------------------
# Pass the noisy test images through the trained model
decoded_imgs = autoencoder.predict(x_test_noisy)

# Plotting
n = 10  # Number of digits to display
plt.figure(figsize=(20, 6))
for i in range(n):
    # Display Original Noisy Image
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test_noisy[i].reshape(28, 28), cmap='gray')
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
    if i == 0:
        ax.set_title("Noisy Images")

    # Display Denoised Reconstruction
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
    if i == 0:
        ax.set_title("Denoised Reconstructions")

plt.show()
```

Output -

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - loss: 0.1664 - val_loss: 0.1179
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - loss: 0.1140 - val_loss: 0.1091
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - loss: 0.1080 - val_loss: 0.1051
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - loss: 0.1046 - val_loss: 0.1025
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - loss: 0.1027 - val_loss: 0.1011
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - loss: 0.1013 - val_loss: 0.0997
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - loss: 0.1003 - val_loss: 0.0989
Epoch 8/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - loss: 0.0994 - val_loss: 0.0985
Epoch 9/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - loss: 0.0988 - val_loss: 0.0978
Epoch 10/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - loss: 0.0983 - val_loss: 0.0972
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step
```



Conclusion -

The implementation of a **Denoising Autoencoder** proved effective in reconstructing clean digits from corrupted images by learning the underlying latent structure of the **MNIST dataset**. The results visually validated that a convolutional bottleneck can successfully filter out random Gaussian noise while preserving essential structural features.